

尾 Warp 归约中的静默错误与 Active Mask 修复

脱敏分享版 | 本文讨论 CUDA warp reduction 的公开编程模型与一个已匿名化的 fused reduction 故障。它与 FTZ sentinel 导致的 CUDA Graph Hang 是两个独立问题。

结论先行

一个 fused AllReduce + RMSNorm kernel 在 block size 不是 32 的整数倍时会产生静默求和错误。根因是尾 warp 只有部分 lane 活跃，却仍使用 `0xffffffff` 作为 shuffle mask；归约读取了 inactive lane 的未定义值。

修复在两个 reduction 点使用 active-lane mask，并保证只累加真实存在的 source lane，不改变 kernel signature、launch config 或 Python API。旧错误在受测 driver、编译组合和输入下可复现，但由于 inactive lane 读取本就没有数值保证，跨 driver 的错误数值与触发率不稳定。修复后，18/18 个代表性 block size 在受测 GPU 单测中达到 bit-exact PASS。

这类 bug 危险之处在于：kernel 通常不会 crash，也不会触发 sanitizer；输出只是偶尔错误，而且是否出现取决于 block size、编译和寄存器残值。

1. Full warp 与 partial warp

NVIDIA warp 通常包含 32 个 lane。若 `blockDim.x` 是 32 的倍数，每个 warp 都完整；否则最后一个 warp 只有一部分线程属于 block：

```
blockDim.x = 36

warp 0: lane 0 ... 31 active
warp 1: lane 0 ... 3 active
        lane 4 ... 31 inactive
```

尾 warp 仍可能执行 warp-level primitive，但 mask 必须准确描述参与线程。把所有 32 位都设为 1，相当于告诉硬件“所有 lane 都在执行且都有合法值”，这与事实不符。

2. 有问题的归约模式

简化后的错误代码类似：

```
unsigned mask = 0xffffffffu;
for (int offset = 16; offset > 0; offset >>= 1) {
    value += __shfl_xor_sync(mask, value, offset);
}
```

在完整 warp 中这可能工作；在 partial warp 中，某些 shuffle 的 source lane 并不存在。CUDA 对这种不满足 mask 合同的行为不提供有效数值保证。

为什么结果可能“有时正确”

- inactive lane 对应寄存器碰巧为 0；

- 编译器版本改变寄存器分配;
- 前序指令留下不同残值;
- 某个 block size 的归约树恰好没有消费危险 lane;
- 两级归约中的错误在后一级偶然抵消。

因此仅测试 128、256、512 等整 warp block size, 完全看不到问题。

3. 正确的 mask 合同

3.1 获取活跃 lane

在所有实际参与归约的线程执行到同一位置后, 可获取:

```
unsigned active = __activemask();
```

如果还有额外谓词, 可使用 ballot 构造参与集合:

```
unsigned participants = __ballot_sync(__activemask(), valid);
```

3.2 不只换 mask, 还要保护 source lane

对连续 active lane 的 down reduction, 概念代码是:

```
unsigned mask = __activemask();
int count = __popc(mask);
int lane = threadIdx.x & 31;

for (int offset = 16; offset > 0; offset >>= 1) {
    float peer = __shfl_down_sync(mask, value, offset);
    if (lane + offset < count) {
        value += peer;
    }
}
```

mask 保证只有参与线程执行 collective, 条件则保证不会消费不存在的 source lane。真实实现还需考虑 lane 集合是否连续、向量值、模板类型和编译器展开。

4. 两级 block reduction 为什么有两个修复点

常见 block reduction 分两级:

```
每个 warp 内归约
-> warp leader 写 shared memory
-> 第一个 warp 归约各 warp partial sum
```

partial warp 问题可能出现在:

1. 最后一个数据 warp 的内部归约;
2. 第一 warp 对 `num_warps` 个 partial sum 的二次归约。

第二级往往也不是完整 32 项，例如一个 block 只有 5 个 warp，就只有前 5 个 lane 持有合法 partial sum。只修第一级仍可能留下同类错误。

5. 最小复现如何设计

5.1 选择能暴露尾 warp 的 block size

测试应刻意包含：

- $32k + 1$ ；
- $32k - 1$ ；
- 不规则中间值；
- 较大但非整倍数的值。

例如 129、159、180、1000。只测常见的 128/256/512 会制造虚假的安全感。

5.2 输入要放大错误

使用全 1、交替符号、极小值、极大值和随机值，并与简单 reference 做比较。全 0 输入通常无法暴露读取未初始化 lane 的影响。

5.3 旧错误对照必须限定环境

未定义行为可以在受测环境中作为 negative control，但不能要求它在不同 driver、编译器或优化选项下复现相同错值。有效的修复证据包括：

- 记录 driver、编译器、输入与 block size，旧实现在该受测组合下可观察到错误；
- 新实现相同输入通过；
- 完整 warp block size 没有回归；
- header/模板联合编译通过；
- API 与 launcher 无变化。

如果换一个 driver 后旧错误暂时没有出现，不能据此认为 full-mask 实现已经正确；相反，这正是未定义数值依赖运行环境的表现。

6. 验证结果

检查	结果
旧错误 negative control	受测 driver/编译/输入组合下 PASS；跨 driver 不保证
非 32 倍数 block-size 矩阵	18/18 bit-exact PASS
两个 reduction 点	均使用 active-lane 语义
Kernel signature	不变
Launch config	不变
Python API	不变

这里的“bit-exact”是对受测 reference 和输入而言，不应外推到所有 dtype、架构或所有 reduction 模式。

7. 为什么 sanitizer 可能抓不到

inactive lane 的 shuffle 并不一定表现为传统 global-memory 越界；它违反的是 warp collective 的参与合同。常见内存检查工具可能报告 0 error，但结果仍错。

因此 warp primitive 的测试应包含：

- active mask 组合；
- partial warp；
- divergence；
- block size 边界；
- 多种编译优化；
- 与独立 reference 的结果比较。

8. 与 FTZ sentinel 问题的区别

问题	Partial-warp reduction	FTZ sentinel
现象	静默数值错误	GPU 轮询不退出 / Graph hang
根因	mask 声称 inactive lane 参与	浮点比较误认控制位模式
修复	active mask + source-lane guard	整数位级 sentinel 比较
主要门禁	非整 warp block-size 数值矩阵	payload 位模式与重复 Graph replay

二者都发生在 fused communication/reduction kernel，但错误机制和验证方法完全不同，不能合并成同一故障叙事。

9. 可复用检查表

- ☐ block size 是否可能不是 warp size 的整数倍？
- ☐ shuffle mask 是否等于真实参与 lane 集合？
- ☐ source lane 不存在时是否仍读取其值？
- ☐ block reduction 的第二级是否同样是 partial warp？
- ☐ 是否覆盖 `32k±1` 和不规则 block size？
- ☐ 是否有独立 reference，而不只依赖 sanitizer？
- ☐ 是否记录 negative control 的 driver、编译器、输入和 block size，并避免要求跨 driver 复现同一错值？
- ☐ 修复是否意外改变 launcher 或 API？

10. 总结

Warp-level primitive 的 mask 是执行协议，不是性能提示。`0xffffffff` 只有在 32 个 lane 都真实参与时才合法；尾 warp 必须使用 active mask，并保护不存在的 source lane。静默错误往往比 crash 更危险，而旧错值本身又可能随 driver 和编译组合变化；因此边界 block size、限定环境的 negative control 和独立 reference 应成为所有 reduction kernel 的固定测试项。